CineBook — SQL Analysis and Build Plan

Based on `00_extensions.sql` to `04_procedures.sql`, plus HANDOVER.md and the project README

Prepared 1 November 2026. Final submission: 10 November 2026 (9 days left).

---

1. What the SQL already gives you

File	What it builds	Count
`00_extensions.sql`	`btree_gist` extension (needed for the overlap rule)	1 extension
`01_tables.sql`	13 tables, all foreign keys, 4 composite FKs on `BOOKING_SEAT`, partial unique index `uq_active_seat_per_show`, index `idx_booking_user_booked_at`	13 tables, 2 indexes
`02_constraints.sql`	CHECKs on role/category/status/method/amounts, `excl_show_no_overlap` (EXCLUDE on screen + time range)	18 CHECKs, 1 EXCLUDE
`03_triggers.sql`	Manager-role check, audit log, seat release, seat count (1 to 6, deferred), confirmation check (deferred)	5 triggers
`04_procedures.sql`	`sp_create_booking`, `sp_confirm_booking`, `sp_cancel_booking`	3 procedures

Important: there is no views file, no seed data, and no search or dashboard SQL yet. Everything that reads data for the app is still to be written.

How the pieces work together (the booking lifecycle)

```
sp_create_booking   -> BOOKING (HELD, 10 min hold) + BOOKING_SEAT rows (is_active = true, price_paid NULL)
        |                 audit trigger logs NULL -> HELD
        v
sp_confirm_booking  -> spreads total over seats (price_paid), inserts PAYMENT (SUCCESS), status -> CONFIRMED
        |                 deferred trigger checks payment amount = total = sum(price_paid)
        v
sp_cancel_booking   -> refund 100% (>24h) or 50% (2h to 24h), PAYMENT -> REFUNDED, status -> CANCELLED
                          release-seats trigger sets is_active = false, so the seat can be booked again
Abandoned hold      -> status -> EXPIRED (only set when the next sp_create_booking runs), same release trigger
```

Which mechanism enforces which rule

Rule	Enforced by
R1 seat unique within screen	composite PK `(screen_id, seat_no)`
R2 no double-booking of an active seat	partial unique index `uq_active_seat_per_show`
R3 no overlapping shows	EXCLUDE constraint with `tstzrange(start, end, '[)')`
R4/R10 price snapshot and discount split	`list_price`, `price_paid`, `sp_confirm_booking`, confirmation trigger
R5 hold, confirm, cancel, expire	`status` CHECK, procedures, expiry sweep inside `sp_create_booking`
R6 cancellation window and refund	`sp_cancel_booking`
R8 manager must have role MANAGER	`trg_venue_manager_role`
R9 audit	`trg_booking_audit`
R11 ticket snapshot	`movie_title`, `venue_name`, `screen_name`, `show_start` on `BOOKING`
1 to 6 seats	deferred constraint trigger, forced early by `SET CONSTRAINTS ... IMMEDIATE`

---

2. Findings from reading the SQL

Ordered by how much they will affect the rest of the build.

Must fix or handle (will cause bugs or break the demo)

1. **A comment refers to an index that does not exist.** `04_procedures.sql` says the expiry sweep is "cheap because of the status index", but `01_tables.sql` only creates `idx_booking_user_booked_at`. Add an index such as `ON BOOKING (hold_expires_at) WHERE status = 'HELD'`.
2. **Expired holds keep seats blocked until someone calls `sp_create_booking`.** A HELD booking only becomes EXPIRED (and its seats freed) inside that procedure. The seat-map read in the backend must treat `status = 'HELD' AND hold_expires_at < now()` as available, or the backend must run the same expiry `UPDATE` before reading. Cleanest fix: add a small `sp_expire_holds()` procedure and call it before every seat-map read.
3. **No check that the show is in the future.** `sp_create_booking` will happily hold seats for a show that already started. Add a check on `v_show_start > now()`. (Keep this in mind for seeding, see Step 2 in section 3.)
4. **No path for a failed payment.** `PAYMENT.status` allows `INITIATED` and `FAILED`, and rule R7 says a retry creates a new payment row, but `sp_confirm_booking` only ever inserts `SUCCESS`. The README also expects the transaction to roll back on a simulated payment failure. Decide the backend behaviour: roll back the confirm, and write a `FAILED` payment row in a separate small transaction (or add `sp_record_failed_payment`).
5. **Missing foreign-key and search indexes.** PostgreSQL does not index foreign keys automatically. Add at least: `MOVIE_SHOW(movie_id)`, `MOVIE_SHOW(start_time)`, `VENUE(city_id)`, `VENUE(manager_id)`, `BOOKING(show_id)`, `BOOKING(coupon_id)`, `PAYMENT(booking_id)`, `BOOKING_AUDIT(booking_id)`. These matter for search and dashboards and for the `EXPLAIN ANALYZE` comparison in Phase 5.
6. **Calling procedures with `INOUT` from Node.** `p_booking_id`, `p_payment_id` and `p_refund_amount` are `INOUT`, so `pg` must pass a placeholder: `CALL sp_create_booking($1,$2,$3,$4,NULL)`. The returned row holds the new id.
7. **`SET CONSTRAINTS ... IMMEDIATE` lasts until the transaction ends.** If the backend runs several procedures in one transaction, later inserts into `BOOKING_SEAT` will be checked immediately too. Run one procedure per transaction, or be aware of this.

Should handle (correctness and polish)

8. **Error codes the backend must translate.** Double booking gives `23505` (unique violation), overlapping show gives `23P01` (exclusion violation), bad value gives `23514` (check), bad reference gives `23503` (foreign key), and all trigger or procedure `RAISE EXCEPTION` messages give `P0001`. Map these to clean HTTP 409/400 responses.
9. **Two simultaneous holds on the same seat.** The second insert waits on the first, then fails with `23505` if the first commits. That is correct behaviour, but the API must return "seat just taken", not a 500.
10. **Duplicate seat numbers in one request** (`ARRAY['A1','A1']`) fail on the primary key. Fine, but validate in the backend first.
11. **Manager rule is one-sided.** The trigger only runs on `VENUE`. Changing a user's role away from MANAGER later is not caught. Acceptable for this scope, worth one line in the report.
12. **Email uniqueness is case-sensitive** (`Ravi@x.com` and `ravi@x.com` are different). Lower-case emails in the backend or add a unique index on `lower(email)`.
13. **Coupons have no per-user limit**, only a global `max_uses`. Fine for the stated rules.
14. **Audit rows have no "changed by".** Fine for the stated rules.

Documentation inconsistencies to clean up

- The README run commands (Section 5) and folder tree (Section 4) omit `00_extensions.sql`. Without it, `02_constraints.sql` fails on the EXCLUDE constraint.
- README Phase 6 says "see Section 5 below" for the folder structure, but the folder structure is Section 4.
- `PROJECT_PLAN_README.md` and `README (1).md` are byte-for-byte identical. Keep one as `README.md` in the repo.
- The README status line says "as of 18 October 2026" while HANDOVER says Phase 1 is complete. Update the status once you confirm what Phase 2 and 3 have actually finished.

---

3. Build plan: what to create, in order

Step 0: Environment (half a day)

- `docker-compose.yml` with PostgreSQL 16, mounting `db/` so the SQL files run in order.
- Create the repo layout from the README, with `db/00_extensions.sql` added.
- Verify on a **fresh** database that files 00 to 04 run with zero errors. Do not skip this, everything else depends on it.

Step 1: Finish the database layer (`db/05_indexes_views.sql`, about 1 day)

Add one new file after `04_procedures.sql`:

- The indexes from finding 1 and 5.
- `sp_expire_holds()` (finding 2) and the future-show check (finding 3).
- Views and queries for the dashboards promised in the report:

Query	Needed for	Technique
Show search by city, movie, language, date, price	`POST /shows/search`	joins on `MOVIE_SHOW`, `MOVIE`, `SCREEN`, `VENUE`, `SHOW_PRICE`
Seat map for a show	seat-selection screen	`SEAT` left-joined to active `BOOKING_SEAT`, treating expired holds as free
Revenue per venue per month	admin dashboard	view over confirmed `BOOKING`
Occupancy per show	manager and admin	booked seats / total seats
Top 3 movies per city	admin dashboard	CTE with `RANK()` or `ROW_NUMBER()`
Peak booking hours	admin dashboard	`GROUP BY EXTRACT(HOUR ...)`
Ticket read path	`GET /bookings/:id`, `GET /me/bookings`	`BOOKING` + `BOOKING_SEAT` only (uses the snapshot columns)

Step 2: Seed data (`db/seed/generate_seed_data.py`, about 1.5 days)

Insert in dependency order: **users first** (managers are needed before venues), then cities, venues, screens, seats, movies, shows, show prices, coupons.

Practical rules learned from the SQL:

- **Shows must not overlap.** Schedule each screen sequentially: `end_time = start + duration_min + a cleaning gap`, then the next start is at or after that end.
- **Every show needs a `SHOW_PRICE` row for every seat category that exists on its screen**, or `sp_create_booking` raises "No price set".
- **Create at least one user per role.** Use a known demo password for the login demo and hash it with bcrypt so the backend's login works.
- **Create bookings through the procedures**, as HANDOVER recommends, so every rule is exercised.
- **Back-dated history:** the procedures stamp `booked_at` with `now()`, so every booking would land on the same day and the monthly revenue and peak-hours dashboards would look empty. After confirming a booking, `UPDATE BOOKING SET booked_at = ...` (no trigger fires on that column). Spread bookings across the last 30 to 60 days and use shows in the past and in the future.
- **Cancellations** only work on future shows more than 2 hours away, so make those on future shows.
- Include a few `HELD`, `EXPIRED`, `CANCELLED` and coupon bookings so the audit table and every status show up.
- Target well above 500 rows total, for example 5 cities, 10 venues, 25 screens, about 2,500 seats, 20 movies, 300 shows, 50 users, 400 or more bookings.

Step 3: W3 constraint test log (`db/tests/w3_tests.sql`, about 0.5 day)

Write one script that tries each bad action and records the error. Expected result for each:

Test	Expected failure
Overlapping show on the same screen	`23P01` exclusion violation
Double-booking the same seat and show	`23505` unique violation (`uq_active_seat_per_show`)
7 seats in one booking	`P0001` from the seat-count trigger
Price of 0 or less	`23514` check violation
Expired coupon	`P0001` "invalid or expired"
Venue manager with a non-manager user	`P0001` from the manager trigger
Cancel less than 2 hours before the show	`P0001` from `sp_cancel_booking`
Cancel after 24h then re-book the same seat	succeeds (proves seat release)

Save the terminal output or screenshots. These go straight into the final report.

Step 4: Backend (Node + Express + `pg`, about 3 days)

Rule: the backend calls the procedures and never re-implements booking logic.

Endpoint	Calls	Notes
`POST /auth/register`, `/auth/login`	`APP_USER`	bcrypt and JWT, role middleware
`POST /shows/search`	search query	filters: city, movie, language, date, price
`GET /shows/:id/seats`	`sp_expire_holds()` then seat-map query	expired holds show as free
`POST /bookings`	`CALL sp_create_booking(..., NULL)`	one transaction, map error codes
`POST /bookings/:id/pay`	`CALL sp_confirm_booking(id, method, NULL)`	simulated failure: roll back, then log a `FAILED` payment
`POST /bookings/:id/cancel`	`CALL sp_cancel_booking(id, NULL)`	returns the refund amount
`GET /bookings/:id`, `GET /me/bookings`	`BOOKING` + `BOOKING_SEAT`	the fast read path
Manager: show scheduling, prices, own revenue	inserts into `MOVIE_SHOW`, `SHOW_PRICE`, views	surface `23P01` as "screen busy at that time"
Admin: CRUD for cities, venues, screens, movies, dashboards	views from Step 1	

Build one shared error handler that turns SQLSTATE codes into clear messages. Test every endpoint in Postman before starting the UI.

Step 5: Frontend (React + Vite + Axios, about 3 days, can overlap with Step 4)

Agree the API contract above first so the frontend can start against it. Pages: login/register, show search, seat map, checkout with coupon, booking history with cancel, manager pages, admin dashboard (simple tables or bar charts are enough). Role-based routing for the three user types. Show the 10-minute hold countdown on checkout.

Step 6: Integration and performance proof (1 day)

- Run everything through Docker Compose.
- Walk through W1 (book, pay, cancel), W2 (dashboards) and W3 (rejections) in the UI.
- `EXPLAIN ANALYZE` the naive 8-table ticket query against the snapshot query (`BOOKING` + `BOOKING_SEAT`). Do it on the seeded data **after** adding the Step 1 indexes, and save both plans and timings for a table or chart in the report.

Step 7: Final report and demo (1.5 days)

Update the mid-sem report with: final schema (note the added indexes and `sp_expire_holds`), SQL snippets, W3 test log, UI screenshots, the `EXPLAIN ANALYZE` comparison. Rehearse the demo once, assign who presents W1, W2 and W3, and confirm that the README alone can set the project up from scratch.

---

4. Suggested schedule from today

The original plan dates have moved, and this plan does not know exactly what Phase 2 and 3 have finished. Treat this as a compressed schedule and tick off anything already done.

Dates	Work	Lead (from the README ownership table)
1 – 2 Nov	Step 0, Step 1, Step 2, Step 3 (DB verified, indexes and views, seed data, W3 log)	Pratham (DB), Arjun and Sushanth (seed, tests, views)
2 – 5 Nov	Step 4 backend, API contract agreed on day 1	Imran, Pratham for DB-heavy endpoints
3 – 7 Nov	Step 5 frontend, starting against the agreed contract	Harinandha, Imran
8 Nov	Step 6 integration and `EXPLAIN ANALYZE`	whole team
9 – 10 Nov	Step 7 report, demo rehearsal, buffer	whole team

If time runs short, cut in this order: frontend polish and charts first, then extra admin CRUD pages. Do not cut: the booking flow end to end, the W3 test log, and the `EXPLAIN ANALYZE` comparison, because those are what demonstrate the database work.

---

5. Definition of done (checklist)

- ☐ Files 00 to 05 run on a fresh database with zero errors
- ☐ Seed script gives 500+ rows, including back-dated and cancelled bookings
- ☐ W3 log shows every bad action rejected with the expected error code
- ☐ Booking, pay, cancel work from the UI and the seat becomes bookable again after a cancel
- ☐ Simulated payment failure rolls back cleanly and is recorded as `FAILED`
- ☐ Dashboards return data from views and CTEs
- ☐ `EXPLAIN ANALYZE` comparison saved with numbers
- ☐ README is corrected (includes `00_extensions.sql`, current status) and the repo is pushed